📝 Multithreading – Basics Notes (Revision)

1️⃣ Process

- A process is an independent program in execution.
- It has its own memory space.
- Created by the Operating System.
- Example: When we run a Java application, one process is created.

2️⃣ Thread

- A thread is a smaller unit of a process.
- It is lightweight.
- Multiple threads can exist inside one process.
- Threads share the same memory of the process.
- Used to achieve concurrent execution of tasks.

3️⃣ Concurrency

- Concurrency is the ability of a system to handle multiple tasks by switching between them so that they make progress together.
- Can happen even on a single CPU core.
- Achieved using context switching.

4️⃣ Parallelism

- Parallelism is the ability of a system to execute multiple tasks at the exact same time.
- Requires multiple CPU cores.

🔥 Key Difference (Very Important for Interview)

- Concurrency = Tasks are in progress together (not necessarily at same time).
- Parallelism = Tasks run at the same exact time.

Now quick retention check 🧠
Without looking:

👉 Do threads have separate memory like processes, or do they share memory?
Ans: Threads **share the same memory space** of the process.

👉 If two threads modify the same variable at the same time, what kind of problem can happen?
Ans: Race condition problem occurs.

1️⃣ Shared Memory

- Threads inside the same process share the same memory space (heap).
- This allows threads to access common variables and objects.
- Because memory is shared, multiple threads can modify the same data.
- Shared memory increases performance but can cause data inconsistency.

2️⃣ Race Condition

- A race condition occurs when multiple threads access shared data simultaneously and at least one thread modifies it.
- The final result depends on the timing of thread execution.
- It leads to unpredictable or incorrect results.
- Example: Two threads executing count++ at the same time.

3️⃣ Atomic Operation

- An atomic operation is an operation that behaves as a single indivisible step.
- It cannot be interrupted by another thread.
- No other thread can see it in a partially completed state.

- Atomic operations prevent race conditions.

4️⃣ Why count++ is Not Atomic

count++ internally performs three steps:

1. Read current value
2. Increment the value
3. Write the updated value back to memory

- Since these are multiple steps, another thread can interrupt between them.
- This can cause a race condition.
- Therefore, count++ is not atomic.

🔥 **Core Connection (Very Important)**

Shared Memory → Possible Race Condition

Race Condition → Happens because operations are not atomic

Solution → Use synchronization (next topic 👀)

🔐 **Synchronization & Locks – Notes**

1️⃣ What is synchronized?

- synchronized is a keyword used to prevent race conditions.
- It allows **only one thread at a time** to execute a synchronized method or block.
- Other threads must wait until the lock is released.
- It ensures thread safety for shared data.

2️⃣ How synchronized Works

- Every object in Java has an internal **monitor lock**.
- A thread must **acquire the lock** before entering a synchronized method/block.
- If the lock is already held, the thread goes into a **blocked state**.
- When the thread exits the synchronized block, the lock is released.

3️⃣ Object-Level Lock

When you write:

```
public synchronized void increment() {
  count++;
}
```

- Lock is acquired on the **current object (this)**
- This is called **Object-level locking**
- Different objects have different locks
- So:
    - obj1.increment() and obj2.increment() do NOT block each other

---

🔥 **Important Connection**

Shared Memory → Race Condition

Race Condition → Because operations are not atomic

Solution → synchronized

synchronized → Uses object-level lock

Now quick retention test 🧠

Without looking:

👉 In a non-static synchronized method, which lock is acquired?

4️⃣ Class-Level Lock

- Applies to **static synchronized methods**.
- Lock is acquired on the **class object (ClassName.class)**.
- Ensures **only one thread across all instances** can execute the static synchronized method at a time.

5️⃣ Key Points

- Object-level locks → per instance, independent execution.
- Class-level locks → global for all instances of the class.
- Helps prevent **race conditions** on shared data.
- Works together with atomic operations to ensure thread safety.

🔥 **Quick Interview Lines**

- Non-static synchronized → **Object-level lock (this)**
- Static synchronized → **Class-level lock (ClassName.class)**
- Race condition occurs when shared data is accessed concurrently without proper locks.
- Atomic operations = indivisible, cannot be interrupted.

📝 **Multithreading – Thread Creation Notes**

1️⃣ Two Main Ways to Create a Thread

1. Extending Thread class

- Create a class that **extends Thread**.
- Override the **run()** method.
- Create an object of your class and call **start()** to run the thread.
- Limitation: **Cannot extend any other class** (Java single inheritance).

Example:

```
class MyThread extends Thread {
  public void run() {
    System.out.println("Thread running");
  }
}

MyThread t = new MyThread();
t.start();
```

---

2. Implementing Runnable interface

- Create a class that **implements Runnable**.
- Override the **run()** method.
- Pass your task to a **Thread object** and call start().
- Advantage: Can **extend another class** at the same time.

Example:

```
class MyTask implements Runnable {
  public void run() {
    System.out.println("Task running");
  }
}

MyTask task = new MyTask();
Thread t = new Thread(task);
t.start();
```

3️⃣ Key Points

- Always use **start()** to create a new thread.
- Calling **run()** directly executes the code in the **current thread**, no new thread is created.
- Runnable is more flexible than extending Thread.

Quick Interview Lines

- "start() → creates a new thread; run() → executes like a normal method in current thread."
- "Use Runnable for better design; Thread extension only when necessary."

❓ What is the difference between Callable and Runnable in Java?

Key Differences

Feature	Runnable	Callable
Package	java.lang	java.util.concurrent
Return Type	void	Generic type <V> (can return a value)
Exception Handling	Cannot throw checked exceptions	Can throw checked exceptions
Method to Implement	run()	call()
Usage with Executor	ExecutorService.submit() returns **Future<?>** but run() returns nothing	ExecutorService.submit() returns **Future<V>** with the result

---

Example:

Runnable

```
Runnable task = () -> System.out.println("Task running");
Thread t = new Thread(task);
t.start();
```

Callable

```
Callable<Integer> task = () -> {
  return 123;
};
ExecutorService executor = Executors.newSingleThreadExecutor();
Future<Integer> future = executor.submit(task);
System.out.println(future.get()); // prints 123
executor.shutdown();
```

🔥 Interview Line to Remember

Use Runnable when you don't need a return value or exception handling.
Use Callable when you need a return value or want to throw checked exceptions.

📝 Thread Lifecycle in Java

Every thread in Java can be in **one of six states**:

1⃣ **New**

- Thread is created using Thread t = new Thread(); or Runnable.
- Thread is **not yet started**.
- t.start() has not been called.

2⃣ **Runnable**

- Thread is **ready to run**.
- Waiting for CPU time to execute.
- After calling t.start(), the thread goes to this state.

3⃣ **Running**

- Thread is actually executing its run() method.
- Only **one thread per CPU core** can be in Running at a time.

4⃣ **Waiting**

- Thread is waiting **indefinitely** until another thread notifies it.
- Happens when using wait() without timeout.

5⃣ **Timed Waiting**

- Thread is waiting for a specified amount of time.
- Happens with methods like sleep(milliseconds), join(milliseconds), wait(milliseconds).

6⃣ **Terminated (Dead)**

- Thread has **finished execution** or stopped.
- Cannot be restarted with start() again.

🔥 **Interview-Friendly Diagram**

New → Runnable → Running → Terminated
↑ ↓
Waiting / Timed Waiting

🔹 **Quick Notes for Interview**

- A thread **starts in New state**, moves to **Runnable** after start().
- From Runnable → **Running** when CPU picks it.
- Thread can go to **Waiting/Timed Waiting** if waiting for resource or sleep.
- **Terminated** → cannot restart the same thread.

❓ After calling t.start(), which thread states are possible for the thread before it finishes execution?

After you call t.start(), the thread **can be in multiple possible states before it finishes**.
Which ones?

- **Runnable** → waiting for CPU to execute
- **Running** → actually executing
- **Waiting / Timed Waiting** → if sleeping or waiting for resource

So the **correct answer** is:
Runnable, Running, Waiting, Timed Waiting

❓ Can a thread **go back** from Running to Runnable? Why or why not?

In Java, a thread can **go from Running → Runnable**.
Why?

- Running doesn't mean the thread has finished.
- The CPU can **preempt** a running thread (time slice over) and put it back into the **Runnable** state to give CPU to another thread.

✅ So **Running → Runnable is possible**.

Quick interview line to remember:
"A running thread may be moved back to Runnable due to CPU scheduling or preemption, even though it hasn't finished execution."

📝 **Waiting vs Timed Waiting (Thread States)**

1⃣ **Waiting**

- Thread is waiting **indefinitely** until another thread **notifies** it.
- Commonly happens when you call:

```
wait();
```

- The thread will **stay waiting forever** unless another thread calls:

```
notify(); // or notifyAll();
```

- Example:

```
synchronized(obj) {
  obj.wait(); // thread waits indefinitely
}
```

2⃣ **Timed Waiting**

- Thread is waiting for a **specified time only**.
- Happens when you call:

sleep(milliseconds);
join(milliseconds);
wait(milliseconds);

- After the time expires, the thread **automatically moves back** to **Runnable**.
- Example:

Thread.sleep(1000); // wait 1 second, then becomes Runnable

🔹 **Key Differences**

Feature	Waiting	Timed Waiting
Duration	Indefinite	Specified time only
Exit Condition	Another thread calls notify/notifyAll	Time expires or notified
Methods	wait()	sleep(ms), join(ms), wait(ms)

🔥 **Interview Line**

"Waiting is indefinite until notification, Timed Waiting is for a specific period and then automatically becomes Runnable."

Quick check for you:

❓ If a thread calls sleep(2000), which state does it enter? And where does it go after 2 seconds?

- When a thread **sleeps**, it enters **Timed Waiting** ✅
- After the time expires, it **moves to Runnable**, **not directly Running**
  - It waits in Runnable until the CPU scheduler picks it to execute → then it moves to **Running**

Correct Flow:

Running → sleep(2000) → Timed Waiting → time over → Runnable → Running (when CPU picks it)

Quick interview line to remember:

"After Timed Waiting, a thread moves to Runnable, then waits for CPU to execute and becomes Running."

📝 **Key Thread Methods in Java**

1️⃣ sleep(long millis)

- Causes the **current thread** to pause execution for a specified time.
- Thread enters **Timed Waiting** during sleep.
- After time expires → moves to **Runnable** → waits for CPU to run.
- **Does not release any locks** if it holds synchronized blocks.

Example:

Thread.sleep(1000); // pauses thread for 1 second

2️⃣ join()

- Makes the **current thread wait** until the thread on which join() is called **finishes execution**.
- Can also take **timeout** → timed waiting.
- Often used to ensure **one thread completes before another starts**.

Example:

t1.start();
t1.join(); // current thread waits until t1 finishes

3️⃣ yield()

- Hint to the **CPU scheduler** that the current thread is willing to **pause** and let other threads execute.
- Thread moves from **Running → Runnable**.
- **No guarantee** which thread will run next.

Example:

Thread.yield();

4️⃣ wait(), notify(), notifyAll()

- **wait()** → Thread enters **Waiting state**, waits indefinitely for notification.
- **notify()** → Wakes up **one waiting thread**.
- **notifyAll()** → Wakes up **all waiting threads**.
- Must be called **inside synchronized block** on the same object.

🔹 **Quick Interview Lines**

- sleep() → timed pause, does **not release locks**
- join() → wait for another thread to finish
- yield() → voluntarily give up CPU, **no guarantee**
- wait/notify/notifyAll() → communication between threads, **must be synchronized**

Quick check for you 🧠

❓ If a thread is inside a synchronized block and calls sleep(1000), does it **release the lock**?
Ans : No

Exactly ✅ — correct!

- When a thread calls **sleep()** inside a synchronized block, it **does NOT release the lock**.
- It just pauses execution, but other threads waiting for the lock **cannot enter** the synchronized block until the sleeping thread finishes or exits the block.

Quick interview line to remember:
"sleep() pauses a thread but **does not release locks**, unlike wait() which releases the monitor lock."

📝 **sleep() vs wait() vs join() in Java**

Feature	sleep()	wait()	join()
Belongs to	Thread class	Object class	Thread class
Purpose	Pause thread for a specified time	Wait for notification from another thread	Wait for another thread to finish
Lock Released?	❌ No	✅ Yes (releases monitor lock)	❌ No
Called inside synchronized?	❌ Not required (but allowed)	✅ Must be inside synchronized block	❌ Not required
State Entered	Timed Waiting	Waiting / Timed Waiting (if timeout)	Waiting / Timed Waiting (if timeout)
Interruptible?	✅ Yes	✅ Yes	✅ Yes
Timeout Available?	✅ Yes (sleep(milliseconds))	✅ Yes (wait(milliseconds))	✅ Yes (join(milliseconds))

🔹 **Interview Line**

- Use **sleep()** → pause thread temporarily
- Use **wait()** → thread waits for a signal, releases lock
- Use **join()** → wait for thread to finish

Quick check for you:

❓ If a thread calls wait() inside a synchronized block, does it release the lock while waiting?
Yes

Exactly ✅ — correct!

- When a thread calls **wait()** inside a synchronized block, it **releases the monitor lock**.
- This allows other threads to enter the synchronized block on the same object.

- Once another thread calls notify() / notifyAll(), the waiting thread moves back to **Runnable** and waits for CPU to run.

Quick interview line to remember:
"wait() releases the lock, sleep() does not." ✅

📝 Thread Priorities, Daemon Threads & Thread Groups

1️⃣ Thread Priority

- Each thread has a **priority** between 1 (MIN_PRIORITY) and 10 (MAX_PRIORITY). Default = 5 (NORM_PRIORITY).
- Higher priority threads are **more likely** to be scheduled before lower priority threads, but **not guaranteed**.
- Methods:

```
t.setPriority(8); // set thread priority
int p = t.getPriority(); // get thread priority
```

Interview line:
Thread priority is a hint to the scheduler; higher priority may run first, but not guaranteed.

2️⃣ Daemon Threads

- Background threads that **support other threads** and **don't prevent JVM from exiting**.
- JVM terminates automatically when **only daemon threads are left**.
- Methods:

```
t.setDaemon(true); // make thread a daemon
boolean d = t.isDaemon(); // check if thread is daemon
```

Common example: Garbage Collector is a **daemon thread**.
Interview line:
Daemon threads run in the background and JVM exits when only daemon threads remain.

3️⃣ Thread Groups

- A **thread group** allows grouping multiple threads together.
- Can **manage threads collectively** (like interrupt all threads in the group).
- Methods:

```
ThreadGroup tg = new ThreadGroup("MyGroup");
Thread t1 = new Thread(tg, task1);
Thread t2 = new Thread(tg, task2);
tg.activeCount(); // number of active threads in the group
tg.interrupt();   // interrupt all threads in the group
```

Interview line:
Thread groups are used for collective management of threads.

🔹 Quick Recap

- **Priority** → scheduling hint
- **Daemon** → background thread, JVM exits automatically
- **Thread Group** → manage multiple threads as a group

Quick check for you 🧠
❓ If a thread is set as a daemon thread, and it's the **only thread left running**, what happens to the JVM?
JVM terminates automatically.
Exactly ✅ — correct!

- If only **daemon threads remain**, JVM **terminates automatically**.
- Non-daemon (user) threads must finish first before JVM exits.

Interview line to remember:

“Daemon threads are background threads; JVM exits automatically when only daemon threads are left.” ✅

📝 Thread Safety in Java

1️⃣ Synchronized Methods & Blocks (we already covered basics)

- Ensure **only one thread at a time** executes critical section.
- Object-level lock → for non-static methods
- Class-level lock → for static methods

Reminder Interview Line:
“Use synchronized to prevent race conditions on shared data.”

2️⃣ ReentrantLock

- Part of java.util.concurrent.locks.
- More flexible than synchronized:
  - Can try to acquire lock with **timeout**
  - Can **interrupt a thread waiting for a lock**
- Example:

```
ReentrantLock lock = new ReentrantLock();
lock.lock();   // acquire lock
try {
   // critical section
} finally {
   lock.unlock();  // release lock
}
```

Interview line:
“ReentrantLock allows more control over locking than synchronized.”

3️⃣ Atomic Classes

- From java.util.concurrent.atomic
- Provide **atomic operations** on variables without explicit synchronization.
- Examples:

```
AtomicInteger count = new AtomicInteger(0);
count.incrementAndGet(); // atomic increment
```

Interview line:
“Atomic classes are useful for simple thread-safe operations without locking.”

4️⃣ Concurrent Collections

- Collections designed for **safe multithreaded access**.
- Examples: ConcurrentHashMap, CopyOnWriteArrayList, ConcurrentLinkedQueue
- Avoids **manual synchronization**.

Interview line:
“Use concurrent collections to handle shared data safely in multithreading.”

🔹 Quick Recap Table

Mechanism	Purpose	Notes
synchronized	Thread safety for critical section	Simple, object/class level lock
ReentrantLock	More flexible locking	Timeout, interruptible, manual unlock
AtomicInteger etc.	Atomic operations	No explicit locks needed
Concurrent Collections	Thread-safe collections	Built-in concurrency handling

---

Quick check for you 🧠

❓ Why would you prefer ReentrantLock over synchronized in a multithreaded Java application?
because of More flexible locking and manual unlock.

Exactly ✅ — that's the main reason!
To expand a bit for interviews:

- **ReentrantLock** allows:
    1. **Try to acquire lock with timeout** → avoid deadlock
    2. **Interruptible lock acquisition** → can stop waiting threads
    3. **Manual unlock in finally block** → better control

Whereas **synchronized**:

- Automatically acquires/releases lock
- Less flexible, cannot interrupt or set timeout

Interview line:
"ReentrantLock provides more flexible control over locks, including timeout, interruptibility, and manual unlock, unlike synchronized." ✅

📝 Deadlock, Livelock & Starvation

1️⃣ Deadlock

- Happens when **two or more threads are waiting indefinitely** for locks held by each other.
- **No thread can proceed**, system gets stuck.

Example:
Thread 1: locks A → wants B
Thread 2: locks B → wants A
// Both wait forever → deadlock
Interview line:
"Deadlock occurs when threads wait on each other indefinitely, preventing progress."

2️⃣ Livelock

- Threads are **active and keep responding**, but **never make progress**.
- Unlike deadlock, threads are not blocked but **continuously retrying**.

Interview line:
"Livelock occurs when threads keep changing state in response to each other but never complete their task."

3️⃣ Starvation

- A thread is **ready to run**, but **never gets CPU time**.
- Happens when **higher priority threads dominate CPU**.

Interview line:
"Starvation occurs when low-priority threads never get CPU because higher-priority threads keep executing."

🔹 Quick Recap Table

Problem	Description	Thread State
Deadlock	Threads wait forever for each other	Blocked/Waiting
Livelock	Threads active but make no progress	Running
Starvation	Threads never get CPU due to low priority	Runnable

---

Quick check for you 🧠
❓ How can we **avoid deadlock** in Java multithreaded programs?
Think of a simple interview answer.
to run the thread in synchrinized way?
Close thinking 👌 — but let's refine it.
Running threads **in synchronized way** can **help avoid race conditions**, but **synchronization alone does not guarantee deadlock prevention**.
Common ways to avoid deadlock:
1️⃣ **Lock Ordering**

- Always acquire multiple locks in the **same global order** in all threads.
- Example:

Thread 1: lock A → lock B
Thread 2: lock A → lock B // same order, avoids deadlock

2️⃣ **Try-Lock with Timeout** (using ReentrantLock)

- Instead of waiting indefinitely, thread tries to acquire a lock with timeout.
- If fails → release acquired locks and retry.

3️⃣ **Minimize Lock Scope**

- Keep synchronized blocks **as short as possible**.
- Avoid holding multiple locks for long periods.

4️⃣ **Avoid Nested Locks**

- If possible, don't lock multiple resources at once.

Interview line:
"Deadlock can be avoided by consistent lock ordering, using tryLock with timeout, minimizing lock scope, and avoiding nested locks." ✅

Quick check for you 🧠
❓ If two threads acquire locks in the **same order**, can deadlock occur?
No
Exactly ✅ — correct!

- If **all threads acquire multiple locks in the same order**, **deadlock cannot occur**.
- This is one of the simplest and most common strategies to prevent deadlocks.

Interview line:
"Consistent lock ordering ensures that threads don't wait indefinitely for each other, preventing deadlock." ✅

📝 **volatile Keyword in Java Threads**

1️⃣ **Definition**

- volatile is a **modifier** for variables.
- Ensures **visibility** of changes made by one thread to **other threads immediately**.
- Without volatile, a thread may **cache variable in CPU register** → other threads may see **stale values**.

2️⃣ **Key Points**

1. **Visibility guarantee**
   - When a thread updates a volatile variable, **all other threads see the latest value** immediately.
2. **Does NOT guarantee atomicity**
   - Example: count++ on a volatile int → **still not thread-safe** (multiple steps: read → increment → write).
3. **Lightweight alternative to synchronized**
   - For **simple flags** or **status variables**, volatile is enough.

3️⃣ **Example**

```
class SharedFlag {
  volatile boolean flag = false;
}

SharedFlag sf = new SharedFlag();

Thread t1 = new Thread(() -> {
  while (!sf.flag) { } // waits until flag becomes true
});
```

```
Thread t2 = new Thread(() -> {
   sf.flag = true; // t1 sees this change immediately
});

t1.start();
t2.start();
```

- Without volatile, t1 **might never see** sf.flag = true.

4⃣ Quick Interview Line

"volatile ensures **visibility** of variable changes across threads, but **does not make operations atomic**." ✅

Quick check for you 🧠

❓ Can we use volatile instead of synchronized for **incrementing a counter** safely? Why or why not

---

📝 **ExecutorService & ThreadPool in Java**

1⃣ Problem

- Creating threads manually (new Thread()) for **every task** → expensive, unscalable.
- Need **thread reuse, scheduling, and resource management** → enter **ExecutorService & ThreadPool**.

2⃣ ExecutorService

- **Interface** in java.util.concurrent that **manages thread execution**.
- Submits tasks without manually managing threads.
- Key methods:

```
ExecutorService executor = Executors.newFixedThreadPool(5);

// submit Runnable (no result)
executor.execute(() -> System.out.println("Task 1"));

// submit Callable (returns Future)
Future<Integer> future = executor.submit(() -> 10 + 20);

// shutdown after tasks
executor.shutdown();
```

✅ Advantages:

- Thread reuse
- Task scheduling
- Can submit **Runnable or Callable**

3⃣ ThreadPool

- A **pool of threads** managed by ExecutorService.
- Common types (via Executors):

1. newFixedThreadPool(int n) → fixed number of threads
2. newCachedThreadPool() → grows/shrinks dynamically
3. newSingleThreadExecutor() → single worker thread
4. newScheduledThreadPool(int n) → supports delayed/repeated tasks

Example:

```
ExecutorService pool = Executors.newFixedThreadPool(3);
for(int i=0;i<10;i++){
   int taskId = i;
   pool.submit(() -> System.out.println("Task " + taskId));
}
pool.shutdown();
```

- **3 threads reused** for 10 tasks → efficient

4⃣ Concurrency Utilities (java.util.concurrent)

- **Locks** → ReentrantLock, ReadWriteLock
- **Atomic Variables** → AtomicInteger, AtomicReference
- **Concurrent Collections** → ConcurrentHashMap, CopyOnWriteArrayList
- **Blocking Queues** → ArrayBlockingQueue, LinkedBlockingQueue
- **CountDownLatch / CyclicBarrier / Semaphore** → thread coordination
- **ExecutorService & ThreadPool** → thread management

5⃣ Java 8 Enhancements

- **Parallel Streams** → process collections in parallel easily

```
List<Integer> nums = Arrays.asList(1,2,3,4,5);
nums.parallelStream().forEach(System.out::println);
```

- **CompletableFuture** → async, non-blocking execution

```
CompletableFuture<Integer> future = CompletableFuture.supplyAsync(() -> 10 + 20);
future.thenAccept(System.out::println);
```

- **ForkJoinPool** → efficient **divide-and-conquer parallelism**

🔹 **Quick Interview Lines**

- **ExecutorService** → manages thread execution, avoids manual thread creation
- **ThreadPool** → reuses threads for multiple tasks → improves performance
- **Java8 Parallel Streams & CompletableFuture** → simplify async and parallel processing
- **Concurrency Utilities** → atomic, thread-safe, coordination mechanisms

---

Quick check 🧠

❓ If you submit 20 tasks to a **FixedThreadPool of 5 threads**, what happens?

Ans:

- **FixedThreadPool of 5 threads** → only **5 threads exist** in the pool.
- **20 tasks submitted** → first 5 tasks are picked up by the 5 threads immediately.
- **Remaining 15 tasks** → **queued in the task queue**, waiting for a thread to become free.
- **As threads finish tasks**, they pick up tasks from the queue **until all 20 tasks are completed**.

✅ Key point: **threads are reused**, no new threads are created beyond the pool size.

Interview line:

"FixedThreadPool executes tasks using a limited number of threads; extra tasks wait in the queue until threads are free."

📝 **Java 8 Concurrency: CompletableFuture, ForkJoinPool, Parallel Streams**

1⃣ CompletableFuture

- **Async, non-blocking task execution**
- Can **chain multiple tasks** and handle **results or exceptions**
- Very useful for **banking operations like transaction processing**

Example: Bank Transaction

```
CompletableFuture<Double> depositFuture = CompletableFuture.supplyAsync(() -> {
  // simulate deposit processing
  System.out.println("Processing deposit...");
  sleep(1000); // pretend delay
  return 1000.0;
});

CompletableFuture<Double> interestFuture = depositFuture.thenApply(amount -> {
```

```
  // calculate interest
  return amount * 0.05;
});

interestFuture.thenAccept(interest ->
  System.out.println("Interest credited: " + interest)
);
```

✅ Key points:

- supplyAsync() → runs task asynchronously
- thenApply() → transforms result
- thenAccept() → consumes result

Banking use case: deposit → interest calculation → send notification

---

2️⃣ ForkJoinPool

- Designed for **divide-and-conquer parallel tasks**
- Breaks large tasks into **smaller subtasks**, executes in parallel, then **combines results**
- Useful for **large batch banking operations** (like calculating total account balances)

Example: Sum of balances

```
class BalanceTask extends RecursiveTask<Double> {
  private double[] balances;
  private int start, end;

  BalanceTask(double[] balances, int start, int end) {
    this.balances = balances; this.start = start; this.end = end;
  }

  @Override
  protected Double compute() {
    if(end - start <= 2) { // threshold
      double sum = 0;
      for(int i=start;i<end;i++) sum += balances[i];
      return sum;
    }
    int mid = (start + end)/2;
    BalanceTask left = new BalanceTask(balances, start, mid);
    BalanceTask right = new BalanceTask(balances, mid, end);
    left.fork();
    double rightResult = right.compute();
    double leftResult = left.join();
    return leftResult + rightResult;
  }
}

// Usage
double[] balances = {1000, 2000, 3000, 4000};
ForkJoinPool pool = new ForkJoinPool();
Double total = pool.invoke(new BalanceTask(balances, 0, balances.length));
System.out.println("Total balance: " + total);
```

3️⃣ Parallel Streams

- Simplifies **parallel processing** on collections
- Automatically uses **ForkJoinPool internally**
- Example: calculate total transaction amount in parallel

```
List<Double> transactions = Arrays.asList(100.0, 200.0, 300.0, 400.0);
double total = transactions.parallelStream().mapToDouble(Double::doubleValue).sum();
System.out.println("Total transactions: " + total);
```

Banking use case: batch processing transactions across multiple accounts efficiently.

🔹 **Quick Interview Lines**

- **CompletableFuture** → async, chainable, non-blocking
- **ForkJoinPool** → divide-and-conquer, efficient parallel tasks
- **Parallel Streams** → easy parallel collection processing
- **Banking Examples:**
  - Process deposit → interest → notification (CompletableFuture)
  - Sum of multiple accounts (ForkJoinPool / Parallel Stream)
  - Batch transactions processing in parallel

Quick check 🧠

❓ If you have **10,000 bank transactions**, which is more efficient:

1. **Loop through transactions sequentially**
2. **Use parallelStream()**

Why?

Please answer it.

Perfect — let's reason it carefully 👌

Scenario: 10,000 bank transactions

1️⃣ **Sequential loop**

- Processes **one transaction at a time**
- CPU cores not fully utilized → slower for large batches

2️⃣ **Parallel Stream**

- Uses **ForkJoinPool internally**
- **Splits transactions across available CPU cores** → multiple transactions processed **at the same time**
- Much faster for **large datasets**

✅ **Answer:**

- **Use parallelStream()** for efficiency.
- Sequential loop is fine for **small datasets**, but for 10,000 transactions, it will be slow.

Interview line:

"Parallel streams efficiently process large datasets by leveraging multiple CPU cores, reducing processing time compared to sequential loops." ✅

If you want, I can now **combine everything we've covered into a "final ultimate multithreading + Java 8 concurrency cheat sheet"** — all in **one-page notebook-ready format** including **banking examples**.

Do you want me to do that?

yes

Perfect ✅ — here's your **ultimate one-page multithreading + Java 8 concurrency cheat sheet**, **interview-ready**, including **banking examples**. You can write this in your notebook and revise in **5 minutes**.

📝 **Java Multithreading & Concurrency – Ultimate Cheat Sheet**

1️⃣ **Threads & Execution**

- **Thread** = lightweight unit of process
- **Concurrency** → tasks make progress by switching
- **Parallelism** → tasks execute at **exact same time** (multi-core)

2️⃣ **Shared Memory & Race Conditions**

- Shared data → race condition if multiple threads modify simultaneously

- **Atomic operation** → indivisible, e.g., AtomicInteger.incrementAndGet()
- count++ → **not atomic** (read → increment → write)

---

3. Thread Safety & Locks

- **Synchronized**
  - Object-level → non-static → lock on object
  - Class-level → static → lock on ClassName.class
- **ReentrantLock** → flexible, timeout, interruptible, manual unlock
- **Atomic classes** → atomic ops without locks
- **Concurrent Collections** → thread-safe (ConcurrentHashMap, CopyOnWriteArrayList)

---

4. Thread Creation

1. **Extend Thread** → override run()
2. **Implement Runnable** → pass to Thread → start()
3. **Callable** → returns value + exception → use ExecutorService + Future

Thread Methods

- sleep(ms) → timed waiting, lock **NOT released**
- join() → waiting for thread, lock **NOT released**
- yield() → hint to CPU
- wait() → releases lock, waits for notify()
- notify()/notifyAll() → wakes waiting threads

---

5. Thread Priority & Daemon

- Priority: 1–10 → scheduling hint
- Daemon → background thread, JVM exits if only daemon threads remain
- ThreadGroup → manage threads collectively

---

6. Thread Pools & ExecutorService

- **FixedThreadPool(n)** → n threads reused
- **CachedThreadPool** → grows/shrinks dynamically
- **SingleThreadExecutor** → single worker
- **ScheduledThreadPool** → delayed/repeated tasks

Example:

```
ExecutorService pool = Executors.newFixedThreadPool(3);
pool.submit(() -> processTransaction(101));
pool.shutdown();
```

Bank use case: reuse threads for processing multiple transactions efficiently

---

7. Java 8 Concurrency

CompletableFuture → async, non-blocking

```
CompletableFuture<Double> depositFuture = CompletableFuture.supplyAsync(() -> deposit(1000));
depositFuture.thenApply(amount -> amount*0.05) // interest
        .thenAccept(interest -> notifyCustomer(interest));
```

ForkJoinPool → divide-and-conquer parallelism

- Large batch operations → total account balance calculation

Parallel Stream → simple parallel processing

```
double total = transactions.parallelStream().mapToDouble(Double::doubleValue).sum();
```

Bank use case: batch transactions, large balance computations, async notifications

---

8. Common Multithreading Problems

Problem	Description	Avoidance
Deadlock	Threads wait each other indefinitely	Lock ordering, tryLock, minimize lock

Problem	Description	Avoidance
Livelock	Active but no progress	Avoid repeated state changes
Starvation	Low-priority thread never runs	Fair locks, balanced priorities

---

9️⃣ volatile Keyword

- Ensures **visibility** across threads
- **Does not guarantee atomicity**

volatile boolean flag = false; // all threads see latest value

---

1️⃣0️⃣ Quick Interview Lines

- Runnable > Thread → more flexible
- Callable → returns value + exception
- sleep() → pauses, does **not release lock**
- wait() → releases lock, waits for notify
- ReentrantLock → flexible, manual unlock
- Object vs Class lock → non-static vs static synchronized
- ParallelStream → large dataset processing using multiple cores

---

💡 **Banking Domain Examples Recap**

1. **Deposit → interest → notify** → CompletableFuture
2. **Sum balances of multiple accounts** → ForkJoinPool or parallelStream
3. **Process 10,000 transactions efficiently** → ThreadPool or parallelStream

📝 Java Multithreading – One-Page Diagram

```
              Thread Lifecycle
+-------+       start()       +---------+     CPU picks     +--------+
|  New  | ----------------------> | Runnable| ------------------> | Running|
+-------+                  +---------+               +--------+
                       |  ^  ^  ^
                       |  | | |
               wait()/sleep()/join() |  |
                       |     |
                       v     |
                   +----------------+
                   | Waiting/Timed  |
                   |  Waiting     |
                   +----------------+
                       |
                       v
                    +---------+
                    |Terminated|
                    +---------+
```

Key Methods & Effects:
- sleep(ms) → Timed Waiting, lock NOT released
- join()/join(ms) → Waiting/Timed Waiting, lock NOT released
- yield() → Runnable, hint to CPU
- wait()/wait(ms) → Waiting/Timed, lock RELEASED
- notify()/notifyAll() → Runnable, wakes waiting threads

Locks:
- Object-level (non-static synchronized) → lock on object (this)
- Class-level (static synchronized) → lock on ClassName.class
- ReentrantLock → flexible: timeout, interruptible, manual unlock
- Atomic classes → thread-safe operations (AtomicInteger, etc.)
- Concurrent Collections → thread-safe shared data

Thread Creation:
1. Extend Thread → override run(), t.start()
2. Implement Runnable → pass to Thread, t.start()
3. Callable → returns value, can throw exception, use ExecutorService + Future

Thread Problems:
- Deadlock → threads wait for each other indefinitely → avoid: lock ordering, tryLock
- Livelock → threads active but no progress
- Starvation → low-priority thread never gets CPU → use fair locks

Priorities & Daemon:
- Priority 1–10 → CPU scheduling hint
- Daemon → background thread, JVM exits if only daemon threads remain
- ThreadGroup → manage multiple threads collectively

📝 Java Multithreading – Quick Revision Sheet

1⃣ Threads & Processes

- **Thread** = smaller, lightweight unit of a process.
- Allows **parallel or concurrent execution**.
- **Concurrency**: multiple tasks make progress by switching.
- **Parallelism**: multiple tasks execute at the **same time** (multi-core).

2⃣ Shared Memory & Race Condition

- **Shared Memory** → multiple threads can access same data.
- **Race Condition** → multiple threads update shared data simultaneously → inconsistent results.
- **Atomic Operation** → indivisible step, cannot be interrupted.
- count++ is **not atomic** → multiple steps (read, increment, write).

3⃣ Synchronization

- **synchronized keyword** → allows only **one thread at a time** in critical section.
- **Object-level lock** → non-static synchronized → this object.
- **Class-level lock** → static synchronized → ClassName.class.
- sleep() **does not release lock**, wait() **releases lock**.

4⃣ Thread Creation

Two ways:

1. **Extend Thread** → override run()
2. **Implement Runnable** → override run() + pass to Thread object

- start() → creates new thread; run() → executes in current thread.
- **Runnable preferred** → more flexible, can extend another class.
- **Callable** → returns a value, can throw checked exceptions.

5⃣ Thread Lifecycle & States

States: New → Runnable → Running → Waiting / Timed Waiting → Terminated

- **Timed Waiting** → sleep(ms), join(ms), wait(ms)
- **Waiting** → wait() until notified
- **Running → Runnable** → possible due to CPU preemption

6⃣ Thread Methods

Method	State/Effect	Notes
sleep(ms)	Timed Waiting	Does not release lock
join()	Waiting / Timed Waiting	Waits for thread to finish
yield()	Runnable	Hint to scheduler
wait()	Waiting	Releases lock, must be synchronized
notify() / notifyAll()	Runnable (wakes threads)	Must be synchronized

---

7⃣ Thread Priorities & Daemon

- Priority: 1–10 → hint to scheduler, not guaranteed.
- Daemon: background threads, JVM exits if only daemon threads remain.
- Thread groups → manage multiple threads collectively.

8⃣ Thread Safety Mechanisms

- synchronized → simple critical section
- ReentrantLock → more flexible, timeout & manual unlock

- Atomic classes → atomic operations, no explicit lock
- Concurrent Collections → thread-safe collections

9⃣ Multithreading Problems

Problem	Description
Deadlock	Threads wait indefinitely for each other
Livelock	Threads active but never make progress
Starvation	Low-priority thread never gets CPU

Deadlock Avoidance:

- Acquire locks in **same order**, use **tryLock with timeout**, minimize lock scope

---

🔹 Quick Interview Lines

- **Non-static synchronized → object-level lock**
- **Static synchronized → class-level lock**
- **Runnable** → more flexible than Thread
- **Callable** → returns value, can throw checked exception
- sleep() → pause thread, **does not release lock**
- wait() → thread waits, **releases lock**
- **ReentrantLock** → flexible locking
- **Daemon thread** → background, JVM exits if only daemon threads remain

---

This sheet covers **everything we discussed** — enough for a **quick 10–15 min revision** before interviews.

If you want, I can also make a **super compact "1-page visual diagram" version** of this sheet with **states, locks, and methods**, so it's **easy to memorize**.
Do you want me to do that too?
Yes do it.
Perfect ✅ — here's a **1-page visual diagram version** of Java Multithreading for quick memorization. You can literally glance at this in 1–2 minutes before interviews.

📝 Java Multithreading – Visual Revision Sheet

```
THREAD LIFECYCLE
+-------+      start()      +---------+      CPU picks      +--------+
|  New  | ------------------> | Runnable| ------------------> | Running|
+-------+                +---------+                +--------+
                       | wait()/sleep()/join()
                       v
                  +----------------+
                  | Waiting/Timed  |
                  |  Waiting     |
                  +----------------+
                       |
                       v
                    +---------+
                    |Terminated|
                    +---------+
```

THREAD CREATION

1⃣ **Extend Thread**

```
class MyThread extends Thread {
   public void run() {}
}
```

```
t.start();
```

2⃣ Implement Runnable

```
class MyTask implements Runnable {
  public void run() {}
}
Thread t = new Thread(new MyTask());
t.start();
```

3⃣ Callable

- Returns value, can throw checked exception
- Used with ExecutorService + Future<V>

THREAD SAFETY & LOCKS

- **Object-level lock** → non-static synchronized → lock on object (this)
- **Class-level lock** → static synchronized → lock on ClassName.class
- **ReentrantLock** → flexible lock: timeout, interruptible, manual unlock
- **Atomic classes** → thread-safe increments (e.g., AtomicInteger)
- **Concurrent Collections** → safe shared data

THREAD METHODS & STATES

Method	State	Lock released?	Notes
sleep(ms)	Timed Waiting	❌	Pause thread
join()/join(ms)	Waiting/Timed	❌	Wait for thread to finish
yield()	Runnable	❌	Suggest CPU switch
wait()/wait(ms)	Waiting/Timed	✅	Must sync, releases lock
notify()/notifyAll()	Runnable	—	Wake waiting threads

PRIORITY & DAEMON THREADS

- **Priority:** 1–10 → CPU scheduling hint
- **Daemon:** JVM exits if only daemon threads remain
- **ThreadGroup:** manage threads collectively

COMMON MULTITHREADING PROBLEMS

Problem	Description	Avoidance
Deadlock	Threads wait for each other	Lock ordering, tryLock, minimize lock
Livelock	Threads active, make no progress	Avoid repeated state changes
Starvation	Low-priority threads never run	Fair locks, avoid high-priority hog

---

QUICK INTERVIEW LINES

- Runnable > Thread → more flexible
- Callable → returns value + exceptions
- sleep() → pauses, does **not release lock**
- wait() → releases lock, waits for notify
- ReentrantLock → flexible, timeout, interruptible
- Object-level vs Class-level lock → non-static vs static synchronized

📝 **Java Multithreading – 1-Minute Memory Cheat (Updated)**

THREAD CREATION

- Thread t = new Thread() → run() / start()
- Runnable → pass to Thread
- Callable → returns value, throws exception (ExecutorService + Future)

THREAD STATES

- **New → Runnable → Running → Waiting/Timed Waiting → Terminated**
- **Timed Waiting:** sleep(ms), join(ms), wait(ms)
- **Waiting:** wait() until notify
- **Running → Runnable possible** (CPU preemption)

THREAD METHODS

- sleep(ms) → Timed Waiting, lock **NOT released**
- join()/join(ms) → Waiting/Timed Waiting, lock **NOT released**
- yield() → Runnable, hint to CPU
- wait()/wait(ms) → Waiting/Timed, lock **RELEASED**
- notify()/notifyAll() → Runnable, wakes threads

LOCKS & SYNCHRONIZATION

- Object-level → non-static synchronized
- Class-level → static synchronized
- ReentrantLock → flexible, timeout, interruptible, manual unlock
- Atomic classes → AtomicInteger, etc.
- Concurrent Collections → thread-safe

THREAD PRIORITY & DAEMON

- Priority 1–10 → CPU scheduling hint
- Daemon → JVM exits if only daemon threads remain
- ThreadGroup → manage multiple threads collectively

VOLATILE KEYWORD

- Ensures **visibility across threads**
- Does **NOT** guarantee atomicity

volatile boolean flag = false;

THREAD POOLS & EXECUTORS

- FixedThreadPool(n) → n threads reused
- CachedThreadPool → grows/shrinks dynamically
- SingleThreadExecutor → single worker
- ScheduledThreadPool → delayed/repeated tasks
- Submit tasks via executor.submit() → Runnable / Callable
- Use shutdown() to stop executor

Banking example: reuse threads to process **10,000 transactions efficiently**

JAVA 8 CONCURRENCY

- **CompletableFuture** → async, chainable, non-blocking
- CompletableFuture.supplyAsync(() -> deposit())
- .thenApply(amount -> amount*0.05) // interest
- .thenAccept(interest -> notifyCustomer(interest));
- **ForkJoinPool** → divide-and-conquer for large batch operations
- **Parallel Streams** → process collections in parallel
- transactions.parallelStream().mapToDouble(Double::doubleValue).sum();

MULTITHREADING PROBLEMS

- Deadlock → threads wait each other → avoid: lock ordering, tryLock
- Livelock → active, no progress
- Starvation → low-priority thread never runs

💡 **Memory Hack**

- Think **"New → Runnable → Running → Waiting/Timed → Terminated"** as core flow
- Attach **methods, locks, volatile, thread pools** to states visually
- Remember: **sleep ≠ releases lock**, **wait = releases lock**